Module – 1

1)Ambiguous Grammar
2)Simplification of Context Free Grammar
3)Reduction of CFG
4)Removal of NULL Production
5)Normal Forms

Ambiguity in context free Grammars: An ambiguous grammar is a grammar in which the same string can be generated in more than one way, giving two or more different parse tree.

OR

A CFG is ambiguous if one or more terminal strings have multiple left most derivation from the start symbol.

(Q.) Show that Grammar is Ambiguous:
G : S → aB|ab
A → aAB|a
B → ABb|b
Solution: In note book

Simplification of Context Free Grammar (CFG):

(Q.) Write a note on CFG Simplification.

Ans: Simplification of Context Free Grammar (CFG):

Simplification of CFG means reducing a given grammar by eliminating unnecessary symbols and production rules while keeping the language of the grammar unchanged.

Sometimes, in a CFG, not all production rules and symbols are required for the derivation of strings. Such useless components are removed to obtain a simpler but equivalent grammar.

The main goal of CFG simplification is:

To make the grammar easier to understand.

To reduce ambiguity and redundancy.

To prepare the grammar for further conversions (like CNF or GNF).

Steps in Simplification of CFG:

Simplification generally involves the following three processes:

i)Reduction of CFG: Removing useless symbols (symbols that do not derive any terminal strings or never appear in sentential forms).

ii)Removal of Unit Productions: Unit productions are of the form $A \rightarrow B$, where both A and B are variables. Such productions are eliminated.

iii) Removal of Null Productions: Null productions are of the form $A \rightarrow \varepsilon$, These are removed carefully while preserving the language.

Reduction of CFG:

Phase 1: Derivation of an equivalent grammar G'.

Here, we make sure each variable derives at least one terminal string.

Derivation procedure:

Step I: Include all symbols w_1 that derive some terminal, and initialize i = 1.

Step II: Include symbols w_{i+1} that derive w_i.

Step III: Increment I and repeat Step II until $w_{i+1} = w_i$.

Step IV: Include all production rules that have w_i in them.

Phase 2: Derivation of an equivalent grammar G''

Here, we ensure that every symbol appears in a sentential form.

Derivation procedure:

Step I: Include the start symbol in y_1 and initialize i=1.

Step II: Include all symbols y_{i+1} that can be derived from y_i and include all production rules that have been applied.

Step III: Increment i and repeat Step II until $y_{i+1} = y_i$

(Q.) Find a reduced grammar equivalent to the grammar G, having production rules:
$P: S \rightarrow Ac|B, A \rightarrow a, C \rightarrow c|Bc, E \rightarrow aA|e$

Solution: Phase I:
Terminal Symbols: T = {a, c, e}
Set w_1 includes all symbols which are derived from terminal symbols:
$w_1 = \{A, C, E\}$
Set w_2 includes all symbols that can derived the symbols present in w_1
$w_2 = \{A, C, E, S\}$
Set w_3 includes all symbols that can derived the symbols present in w_2
$w_3 = \{A, C, E, S\}$
New Grammar G' = {(A, C, E, S), (a, c, e), P, S}
G = {N, T, P, S}
Where N = non – terminal
T = Terminal
P = production Rule
S = Start Symbol
$P: S \rightarrow AC, A \rightarrow a, C \rightarrow c, E \rightarrow aA|e$

Phase II:

y_1 includes start symbol:

$$y_1 = \mathrm{S}$$

y_2 includes all symbols that can be derived from start symbols:

$$y_2 = \{\mathrm{S}, \mathrm{A}, \mathrm{C}\}$$

y_3 includes all symbols that can be derived from y_2:

$$y_3 = \{\mathrm{S}, \mathrm{A}, \mathrm{C}, \mathrm{a}, \mathrm{c}\}$$

y_4 includes all symbols that can be derived from y_3:

$$y_4 = \{\mathrm{S}, \mathrm{A}, \mathrm{C}, \mathrm{a}, \mathrm{c}\}$$

G" = {(A, C, S), (a, c), P, S}

$\mathrm{P}: \mathrm{S} \rightarrow \mathrm{AC}, \mathrm{A} \rightarrow \mathrm{a}, \mathrm{C} \rightarrow \mathrm{c}$

Removal of NULL Production:

(Q.) Remove NULL production from the following grammar:

$S \rightarrow ABAC, A \rightarrow aA|\varepsilon, B \rightarrow bB|\varepsilon, C \rightarrow c$

Solution: First find out NULL production, $A \rightarrow \varepsilon, B \rightarrow \varepsilon$

These two NULL production we have to remove.

a) To Eliminate $A \rightarrow \varepsilon$

$S \rightarrow ABAC$

$S \rightarrow ABC|BAC|BC$

$A \rightarrow aA$

$A \rightarrow a$

New Production is:

P: $S \rightarrow ABAC|ABC|BAC|BC$

$A \rightarrow aA|a$

$B \rightarrow bB|\varepsilon$

$C \rightarrow c$

b) To Eliminate B → ε

We have to check new production where we find B on right side of any production that replace with ε.

S → ABAC|ABC|BAC|BC

Replace B with ε

We get,

S → AAC|AC|C

B → b

New Production is:

P: S → ABAC|ABC|BAC|BC|AAC|AC|C

A → aA|a

B → bB|b

C → c

This is our result.

Normal Forms:

(Q.) Write a note on Normal Forms.

Solution: Normal Forms:

Definition: A context free grammar is said to be in Normal forms of all productions follow some fixed restrictions or rules. In other words, Normal Forms are standard forms of writing productions in CFG to simplify grammar and make parsing easier.

There are two types of Normal Forms:

i) Chomsky Normal Forms (CNF): A Context free grammar is in Chomsky Normal Form if every production is of the form: $A \rightarrow \alpha$ (Where A is a non – terminal and a is a terminal) $A \rightarrow$ BC (Where A, B, C are non – terminal) Right hand side can have only one terminal or exactly two variables.

Example: $S \rightarrow AB|b, A \rightarrow a, B \rightarrow b$

So, this grammar is in CNF.

ii) Greibach Normal Forms (GNF): A context free grammar is said to be in Greibach Normal Form (GNF) if every production rules is of the form: $A \rightarrow a\alpha$ (Where A = a non – terminal, a = a single terminal symbol, $\alpha = \text{string of non} - \text{terminal}$

Example: $S \rightarrow aA|b, A \rightarrow aS|b$

Since every rule begins with a terminal, So this grammar is in GNF.

Module – 2

1) Turing Machine (Design a TM, Construct a Language by TM, Variants of TM)
2) Linear Bound Automata and Automaton Model.
3) Pushdown Automata
4) Church-Turing Thesis
5) Universal TM
6) Recursive Enumerable language
7) Halting Problem
8) Complexity Hierarchy
9) Big – O Notation

Turing Machine

Q) Explain the working of a Turing Machine to print the symbol “011” on a blank tape.

Draw a neat labelled diagram showing each step of the head movement.

Also write the instructions used for bit inversion (0 to 1 and 1 to 0).

Turing Machine: Alan Turing introduced a mathematical model called the Turing Machine. It is the theoretical model of a modern computer. A Turing Machine is used to recognize formal languages and to simulate any computational problem or algorithm. Formula of Turing Machine is: TM = FA + Tape.

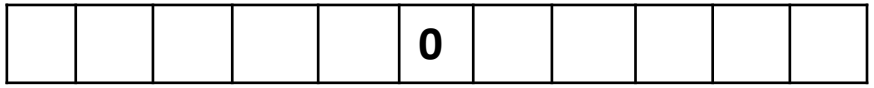
Working of Turing Machine: i) Read the symbol on the tape under the head. ii) Change the state according to the transition function iii) Replace old symbol with a new symbol iv) Move the head either to the Left or right.

A simple diagrammatic representation for Turing Machine construction as follows:

Let’s print the symbol 011 on initially blank tape.

Head

First, we write 0 in Head



Head

Second, we move the tape left by Head

Head

Third, we right 1 for Head

Head

Fourth we move left and insert 1

Head

Let’s see another one program for bit inversion in above tape.

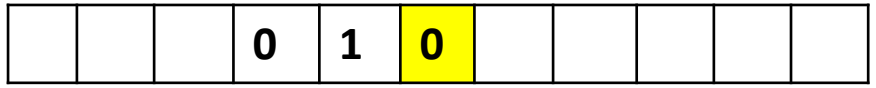
Simple instructions for program:

Read Symbol	Instructions to write	Instructions to move
Blank	None	None
0	Write 1	R
1	Write 0	R

The machine checks head and reads the symbol under the read/write new symbol and move left or right. Let’s see it step by step:

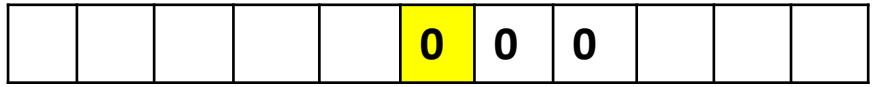
Head

Change the head symbol by 0



Head

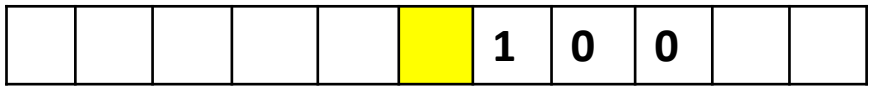
Move right and check the head its again 1



Head

Symbol read by head and repeat the same instruction

Head



Head

When the blank symbol reads machine will be in the halt state.

(Q.) Construct a Turing Machine accepting following language.
$L = \{a^n b^m c^{m+n} | m, n \geq 0\}$

Solution: Language L = {ε, ac, bc, aacc, bbcc, abcc}

1. First scan leftmost a or b and make it α using state q_0.
2. Scan rightmost c and make it y.
3. Find out rightmost x
4. Repeat step 1 to 3 till all a's, b's and c's are not over.
5. Check whether there is extra a or b or c.

Turing machine is given below

$Q = \{q_0, q_1, q_2, q_3, q_{Acc}\}$

$\Sigma = \{a, b, c\}$

$\Gamma = \{a, b, c, x, y, B\}$

δ = Transition function.

Transition table

State	a	b	c	x	y	B
q_0	(q_1, x, R)	(q_1, y, R)	-	-	(q_4, y, R)	-
q_1	(q_1, a, R)	(q_1, b, R)	(q_1, c, R)	-	(q_2, y, L)	(q_2, B, L)
q_2	-	-	(q_3, y, L)	-	-	-
q_3	(q_3, a, L)	(q_1, b, L)	(q_1, c, L)	(q_0, x, R)	-	q_{acc}
q_4	-	-	-	-	(q_4, y, R)	q_{acc}
q_{acc}	-	-	-	-	-	Final

Instantaneous description for abcc $\rightarrow$ Babcc BB

$\rightarrow$ B $\mathbf{q_0}$ **a** b cc B $\rightarrow$ B X $\mathbf{q_1}$ **b** cc B $\rightarrow$ B X b $\mathbf{q_1}$ **c** cB

$\rightarrow$ B x b c $\mathbf{q_1}$ **c** B $\rightarrow$ B x b cc $\mathbf{q_1}$ **B** $\rightarrow$ B x b cc $\mathbf{q_2}$ **B**

$\rightarrow$ B x b **c** $\mathbf{q_3}$ y B $\rightarrow$ B x **b** $\mathbf{q_3}$ c y B

$\rightarrow$ B **x** $\mathbf{q_3}$ bc y B $\rightarrow$ B x q_0 b c y B

$\rightarrow$ B x x $\mathbf{q_1}$ **c** y B

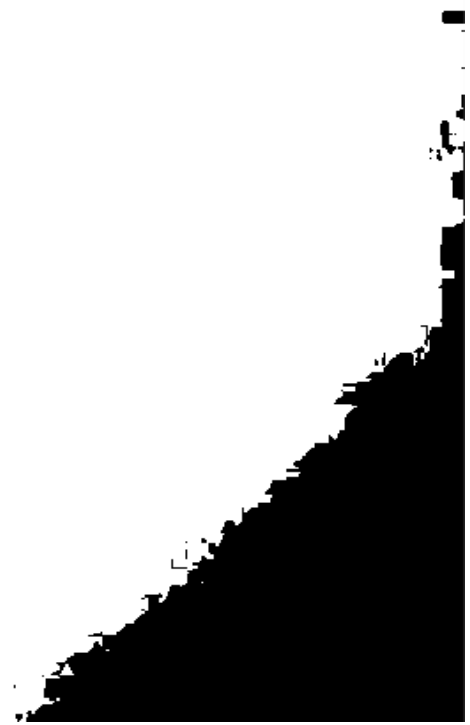
$\rightarrow$ B xx c $\mathbf{q_1}$ **y** B

$\rightarrow$ B xx **c** $\mathbf{q_2}$ y B

$\rightarrow$ B x **x** $\mathbf{q_3}$ yy B

$\rightarrow$ B xx y q_3 y B

$\rightarrow$ B xx yy q_3 B $\rightarrow$ q_{accept}



(Q.) Design a Turing Machine for language.

$L = \{WW^R \mid W \in (0+1)^R\}$ Where W^R is reverse of W.

Solution:

Language for above Turing machine is

$L = \{00, 11, 0110, 1001, 001100, 100001...\}$

We can apply simple logic on this language.

1. If initial state q_0 read 0, then last state also read 0.
2. If initial state q_0 read 1, then last state also read 1.

Turing machine is :

$Q = \{ q_0, q_1, q_2, q_3, q_4, q_5, q_{Acc}\}$

$\Sigma = \{0, 1\}$

$\Gamma = \{0, 1, x, B\}$

Where δ is

Step	0	1	x	B
q_0	(q_1, x, R)	(q_4, x, R)	(q_{acc}, x, R)	-
q_1	$(q_1, 0, R)$	$(q_1, 1, R)$	(q_2, x, L)	(q_2, B, L)
q_2	(q_3, x, L)	-	-	-
q_3	$(q_3, 0, L)$	$(q_3, 1, L)$	(q_0, x, R)	-
q_4	$(q_4, 0, R)$	$(q_4, 1, R)$	(q_5, x, L)	(q_5, B, L)
q_5	-	(q_3, x, L)	-	-
q_{acc}	Final state	-	(q_{acc}, x, R)	(q_{acc}, B, R)

Consider string 0110

ID for B 0 11 0 BB

$\rightarrow$ B q_0 0 110 B $\rightarrow$ B x q_1 110 B B

$\rightarrow$ B x 1 q_1 10 B $\rightarrow$ B x 110 q_1 BB

$\rightarrow$ B x 11 q_2 0 BB $\rightarrow$ B x 1 q_3 1x B B

$\rightarrow$ B x q_3 11 x B B $\rightarrow$ B q_3 x 11 x B B

$\rightarrow$ B q_0 11 x BB $\rightarrow$ B x x q_4 1x Bb

$\rightarrow$ Bxx 1 q_4 x B $\rightarrow$ Bxx q_5 1xB

$\rightarrow$ B x q_3 xxx B

$\rightarrow$ B xx q_0 xx B $\rightarrow$ B xxx q_{acc} x B

$\rightarrow$ B xx xx q_{acc} B $\rightarrow$ B xxxx q_{acc}

$\rightarrow$ q_{acc} = accept the string

Variants of Turing Machine:

Introduction to Variants of Turing Machine:

The Turing Machine (TM), proposed by Alan Turing in 1936, is the most powerful abstract model of computation. It is used to study what problems can or cannot be solved by an algorithm.

To explore the nature of computation more deeply, researchers developed several variants of the Turing Machine by modifying its structure (such as adding multiple tapes, multiple heads, non-determinism, randomness, or even quantum behavior).

These variants help us:

- Compare computational power of different models.
- Study time and space efficiency.
- Understand new paradigms like probabilistic and quantum computation.

Although these variants may differ in efficiency, most are equivalent in power to the basic Turing Machine, meaning they recognize the same class of languages (Recursively Enumerable).

Different Types of Variants of Turing Machine are as follows:

- Multitrack Turing Machine
- Multitape Turing Machine
- Non-deterministic Turing Machine
- Multistack Turing Machine
- Counter Machine

Multi-Track Turing Machine:

•A single tape is divided into multiple tracks.

•Each track holds one symbol, and the head reads/writes across all tracks simultaneously.

•Useful for storing multiple information streams in parallel.

	B	a	b	a	b	B	

Finite control head single TM

	B	a	b	B	B	
	B	b	c	B	B	
	B	B	B	B	B	

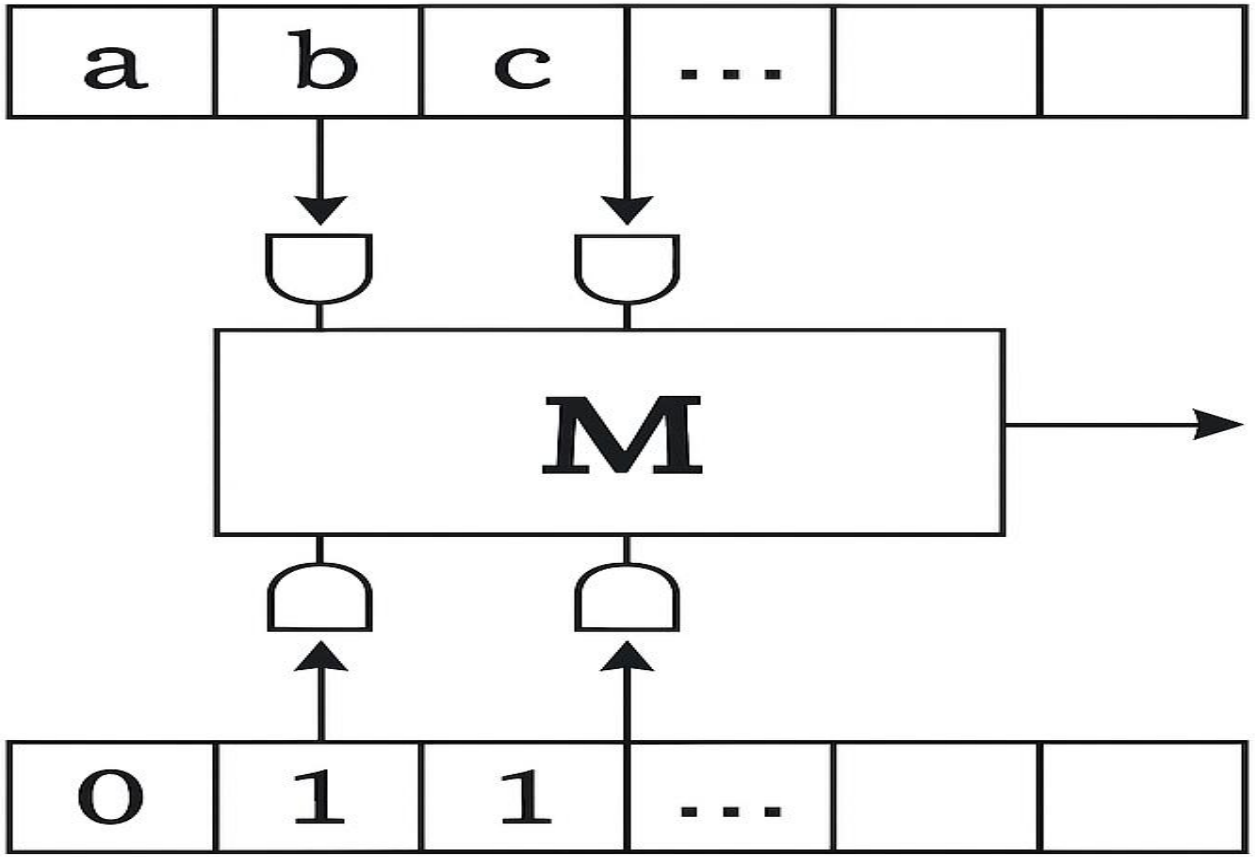
Head multitrack TM

Multi-tape Turing Machine:

•A Turing Machine with more than one tape.

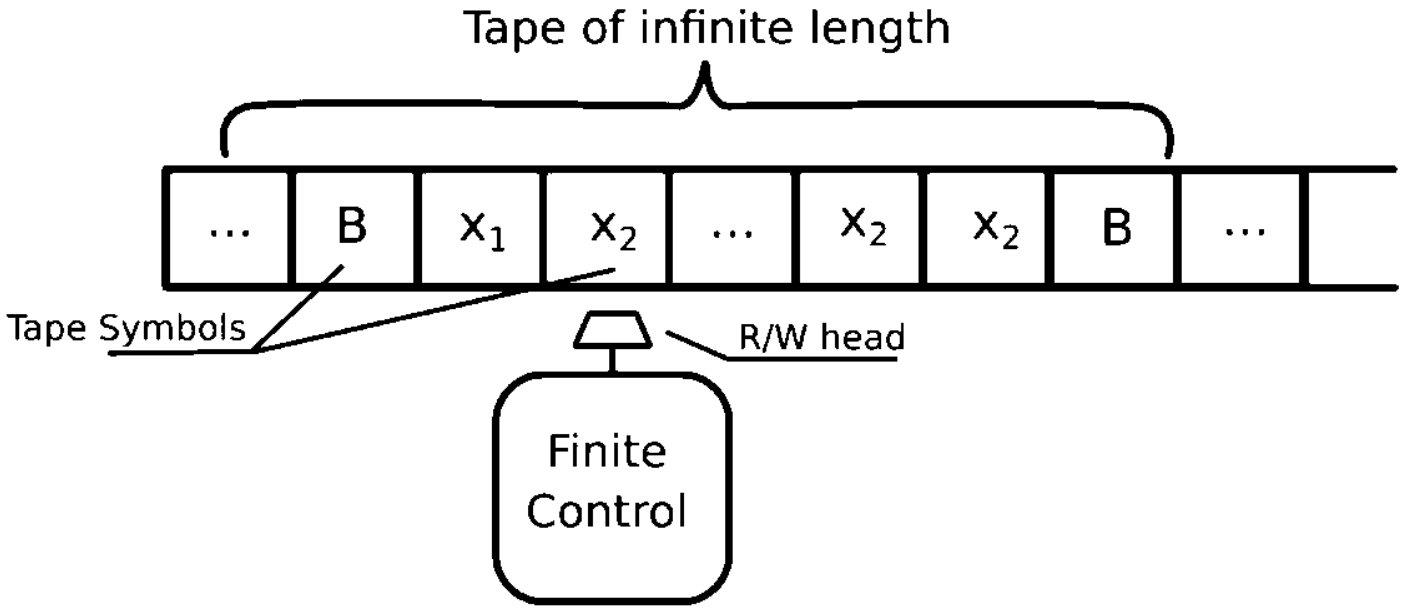
•Each tape has its own read/write head.

•In one step, the machine reads all heads, writes new symbols, moves each head, and changes state.



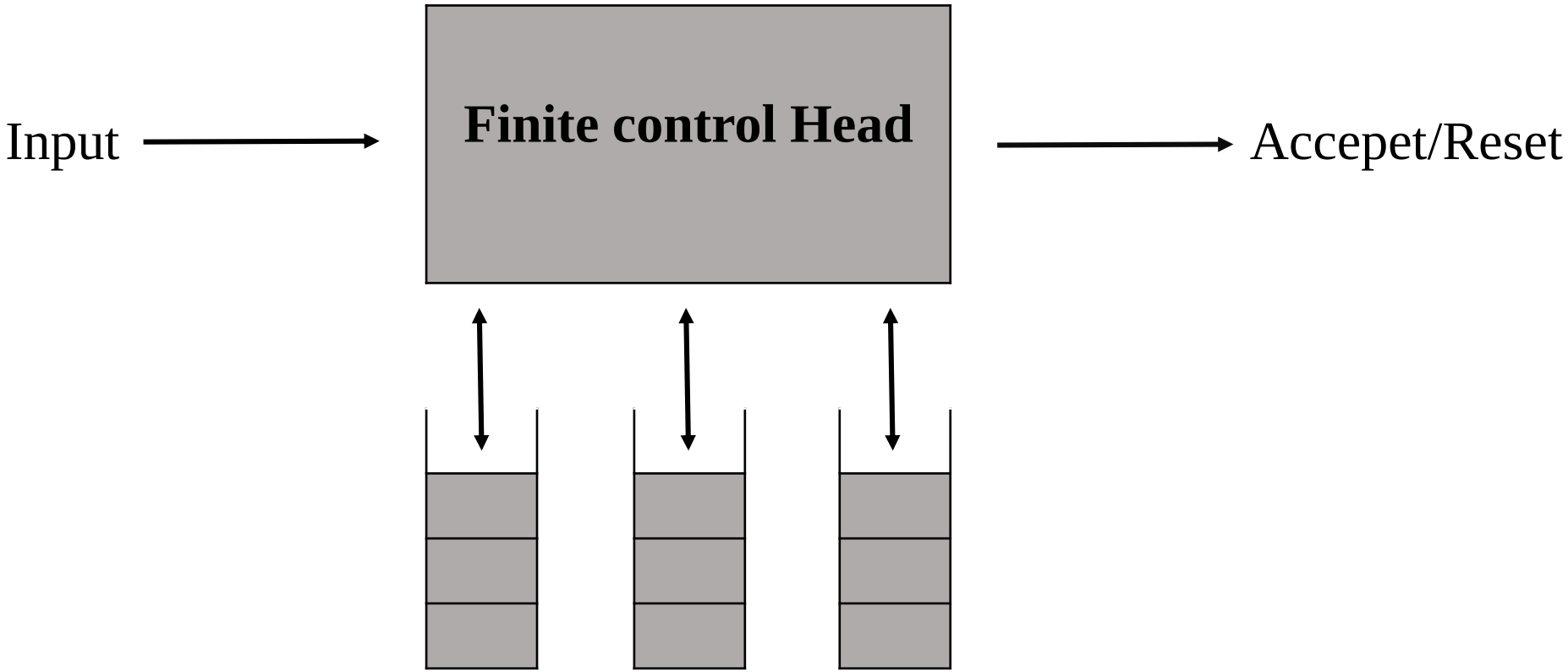
Non-deterministic Turing Machine:

At each step, instead of one possible move, the machine can choose many possible moves.
It's like the machine "branches" into different possibilities at the same time.
A string is accepted if at least one branch leads to an accepting state.
Example: Checking if a string has some property can be done by guessing and verifying.



Multi-Stack Turing Machine:

Similar to a Pushdown Automaton (PDA) but with multiple stacks instead of one.
A single-stack PDA = recognizes Context-Free Languages.
A two-stack PDA = equivalent to a Turing Machine (can simulate memory like a tape).
Multi-stack machines are mainly theoretical models to show equivalence with TM.



Counter Machine:

A restricted form of TM that uses a finite control plus a small number of counters (instead of tapes).

A counter stores a non-negative integer.

Allowed operations:

- Increment counter
- Decrement counter
- Test if counter = 0

A machine with two counters is as powerful as a Turing Machine.

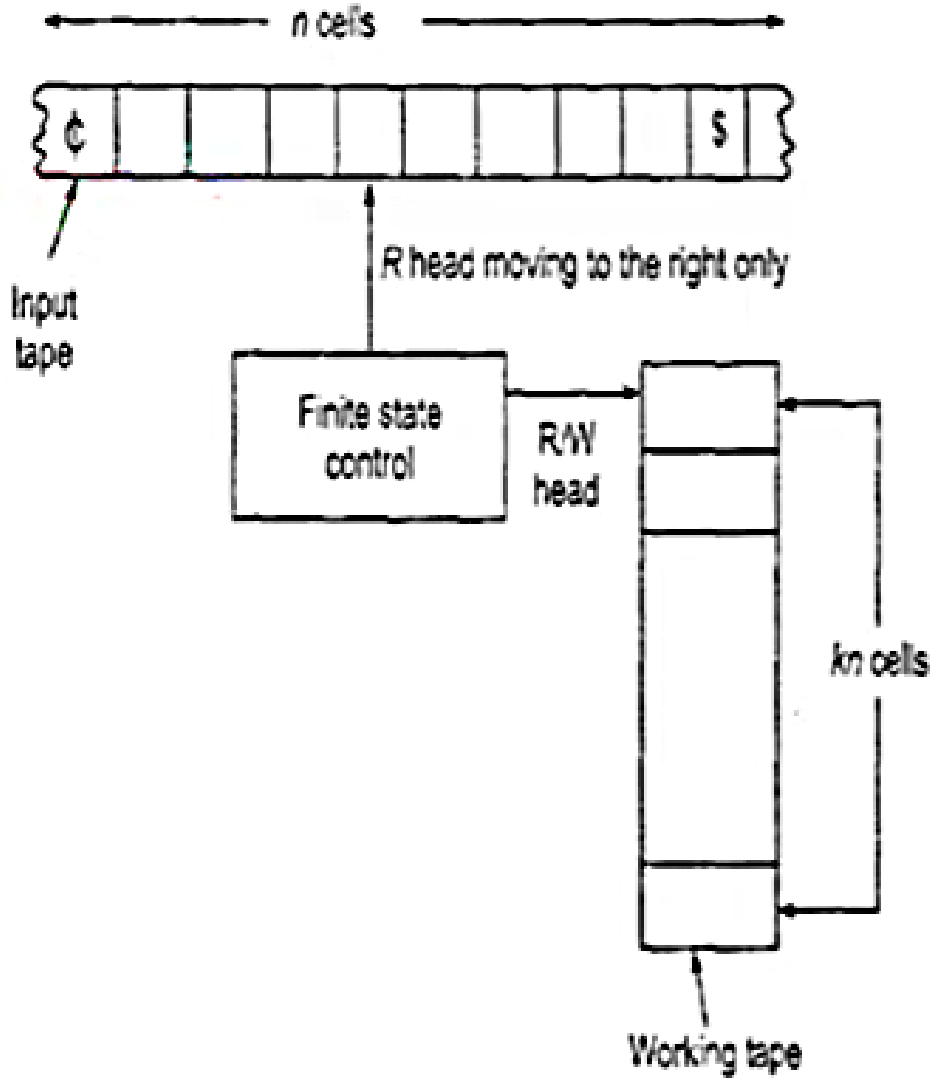
Used in proofs (to show power of very simple models).

Linear Bound Automata & Linear Bound Automaton Model:

Linear Bounded Automata (LBA): A Linear Bounded Automaton (LBA) is a special type of Turing Machine with restricted tape length. In a normal Turing Machine, the tape is infinite, but in an LBA, the tape is linearly bounded by the size of the input. This means that if the input length is n, then the LBA can use at most kn tape cells (for some constant k). LBA accepts the class of languages called Context-Sensitive Languages (CSL).
So, LBA = Turing Machine + Tape bounded by input length.

Linear Bounded Automaton Model: The model of an LBA is written as:
M = $(Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej})$
Where:
- **Q** → finite set of states
- **Σ** → input alphabet
- **Γ** → tape alphabet ($\Sigma \subseteq \Gamma$)
- **δ** → transition function (controls movement L/R, read/write)
- $\mathbf{q_0}$ → initial state
- $\boldsymbol{q_{acc}}, \boldsymbol{q_{rej}}$ → final states

The model of LBA



- Σ contains two special symbols ¢ and $.
- ¢ is called the left-end marker which is entered in the leftmost cell of the input tape and prevents the RIW head from getting off the left end of the tape.
- $ is called the right-end marker which is entered in the rightmost cell of the input tape and prevents the R/W head from getting off the right end of the tape.

Languages accepted by Linear Bound Automaton:

A Linear Bounded Automaton (LBA) is a non-deterministic Turing Machine whose tape is restricted to a portion bounded by the length of the input. The class of languages accepted by an LBA is called Context-Sensitive Languages (CSL). These languages are more powerful than Context-Free Languages but less powerful than Turing-recognizable languages. A Linear Bounded Automaton (LBA) is a type of Turing Machine.
The difference is: A normal Turing Machine can use infinite tape. But an LBA can use only the part of tape which is equal (linearly bounded) to the length of input string.

Languages Accepted:

- The set of all languages accepted by an LBA is called Context-Sensitive Languages (CSL).
- Context-Sensitive Grammar generates these languages.
- In such grammars, rules never reduce the length of string.
- Example: A → aB (length same or increases).

3. Examples of Context-Sensitive Languages (LBA can accept these):

1. **Equal number of a's, b's, and c's**

$$L = \{a^n b^n c^n \mid n \geq 1\}$$

   - Example strings: `abc`, `aabbcc`, `aaabbbccc`.
   - This language is **not** context-free (PDA can't accept), but LBA can.

2. **Palindrome Language**

$$L = \{ww^R \mid w \in \{a, b\}^*\}$$

   - Example: `abba`, `abcba`.

3. **Language of equal a's and b's with restrictions**

$$L = \{a^n b^n c^m \mid n, m \geq 1\}$$

Pushdown Automata:

Definition of Pushdown Automata (PDA): A Pushdown Automaton (PDA) is a type of automaton that works like a finite automaton but has an additional memory called a stack.

• The stack provides unlimited storage in a last-in–first-out (LIFO) manner.

• Because of the stack, a PDA can recognize Context-Free Languages (CFLs).

Formally, a PDA is defined as a 7-tuple:

$M=(Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

where:

Q = set of states

Σ = input alphabet

Γ = stack alphabet

δ = transition function

q_0 = initial state

Z_0 = initial stack symbol

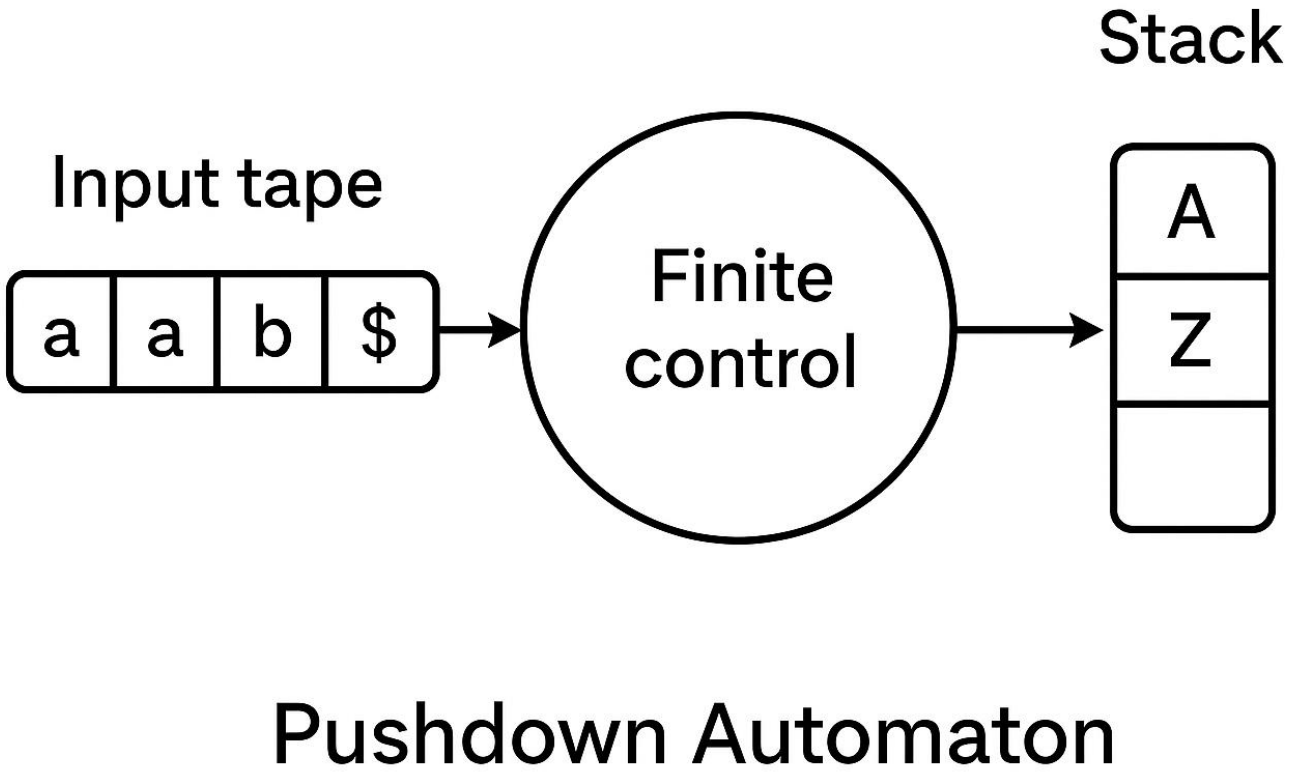
F = set of final states

A stack has two operations namely PUSH operation and POP operation.

- Push Operation: It is used to insert an element into stack.
- Pop Operation: It is used to delete an element from stack.
- Top Operation: Which point to the current input of stack.

PDA has a 3 components:

1) An input tape.
2) A control unit.
3) A stack with infinite size.



(Q.) Construct a PDA that accepts $L = \{0^n, 1^n \mid n \geq 0\}$.

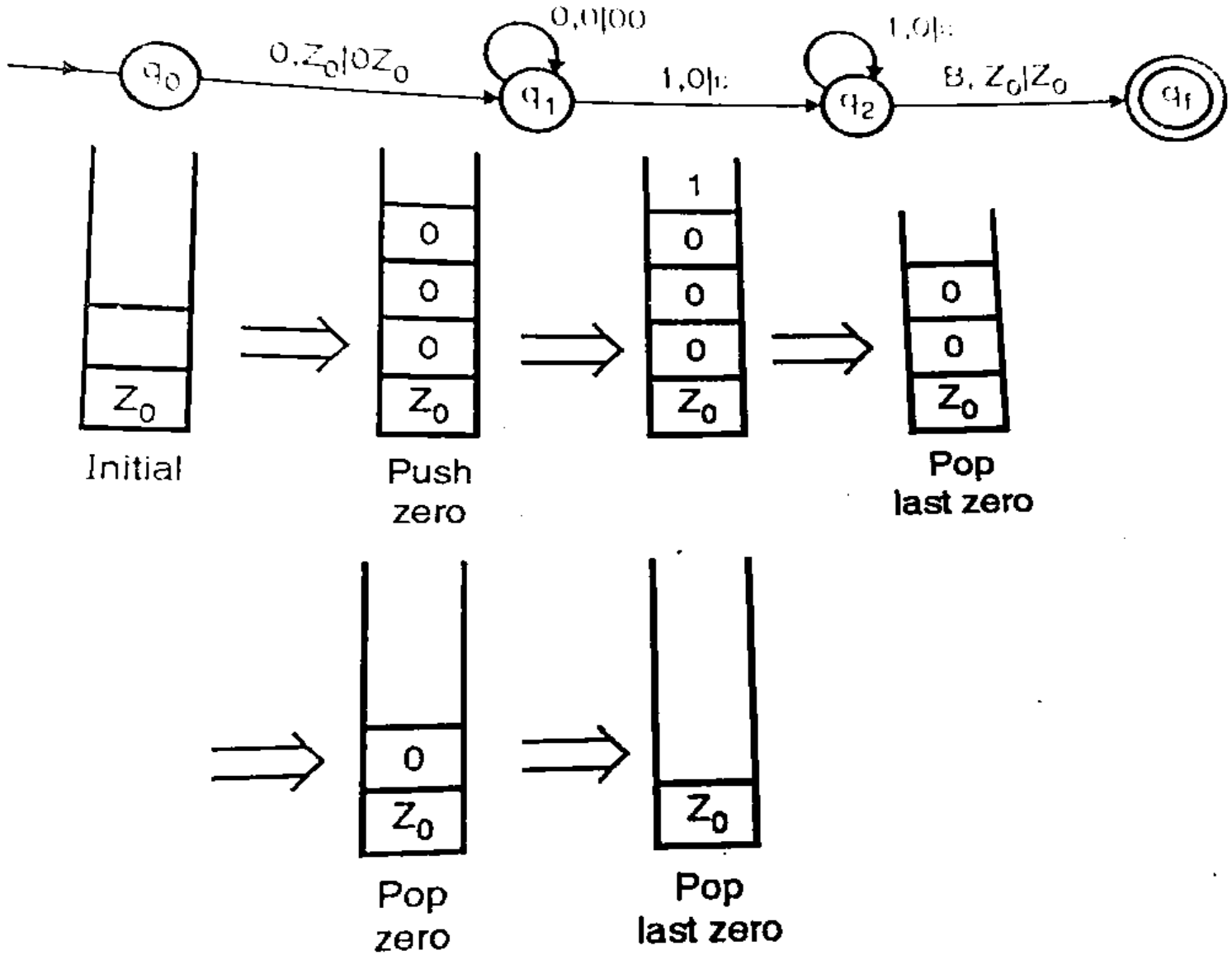
Step:1 Problem Description: 1) If the input symbol is 0 and the stack top is empty, push 0 into the stack. This step may be repeated.

2) If the input symbol is 1 and the stack top is 0, then pop 0 from the stack. This step may be repeated.

3) The process continues until the blank symbol and the initial stack symbol z_0 are encountered.

4) If the stack becomes empty, the string is valid; otherwise the string is invalid.

Step:2 Construct transition diagram for PDA:



Step:3 Transition table for PDA:

Transition	Operation Performed
$q_0 : \delta (q_0, 0, 20) \rightarrow (q_1, 0z0)$	push o
$q_1 : \delta (q_1, 0, 0) \rightarrow (q_1, 00)$	push o
$\delta (q_1, 0, 1) \rightarrow (q_2, \varepsilon)$	pop 0
$q_2 : \delta (q_2, 1, 0) \rightarrow (q_2, \varepsilon)$	pop 0
$\delta (q_2, B, Z_o) \rightarrow (q_f, Z_o)$	Accept
q_f : Accept	

(Q). Construct a PDA which is equivalent to the following given CFG:

$S \to 0CC, C \to 0S, C \to 1S, C \to 0$ Test whether 010^4 is accepted by $N(A)$.

Solution: Step 1: The target PDA is as following:

$A = (\{q\}, \{0, 1\}, \{S, C, 0, 1\}, \delta, q, s, \varphi)$

δ is defined by the rules:

R_1: $\delta(q, \varepsilon, S) = \{(q, 0CC)\}$

$\delta(q, \varepsilon, C) = \{(q, 0S), (q, 1S), (q, 0)\}$

R_2: $\delta(q, 0, 0) = \{(q, \varepsilon)\}$

$\delta(q, 1,1) = \{(q, \varepsilon)\}$

Step 2: Check if 010^4 can get from this production,

$S \Longrightarrow 0CC$

$\Longrightarrow 01SC$

$\Longrightarrow 010CCC$

$\Longrightarrow 010000$

$\Longrightarrow 010^4$

Step 3: Construct PDA

$(q, 010^4, S) \vdash (q, 010^4, 0CC)$

$\vdash (q, 10^4, CC)$

$\vdash (q, 10^4, 1SC)$

$\vdash (q, 0^4, SC)$

$\vdash (q, 0^4, 0CCC)$

$\vdash (q, 0^3, CCC)$

$\vdash (q, 0^3, 0CC)$

$\vdash (q, 0^2, CC)$

$\vdash (q, 0, C)$

$\vdash (q, 0, 0)$

$\vdash (q, \varepsilon, \varepsilon)$

Thus $010^4 \epsilon$ N(A).

Church-Turing Thesis: The Church–Turing Thesis is a fundamental principle of computer science and mathematics. It was proposed independently by Alonzo Church and Alan Turing in the 1930s.

It states that:

Any function which is effectively computable (i.e., can be calculated by a human using a finite set of rules and finite time) can be computed by a Turing Machine.

In other words, whatever can be computed by any algorithm can also be computed by a Turing Machine. This shows that the Turing Machine is a universal model of computation.

Key Points:

Effectively Computable → A task that can be solved by step-by-step procedure (algorithm).

Equivalence → Church used the concept of lambda calculus and Turing used Turing Machine, but both gave the same power of computation.

Foundation → It provides the basis for theoretical computer science and explains the limits of what machines can compute.

Importance → It connects the idea of algorithms, mathematics, and machines into one theory.

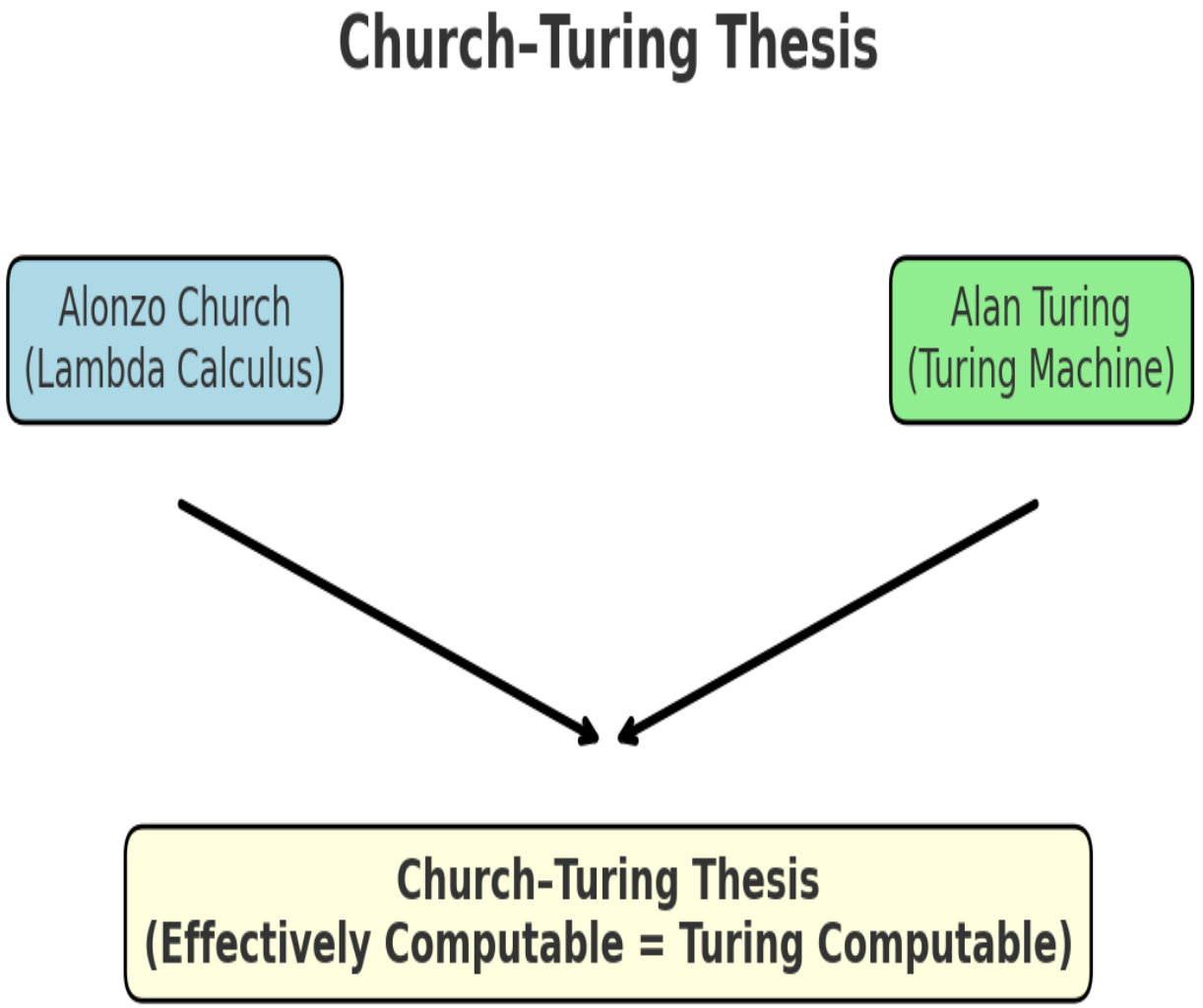
Explanation of Diagram:

In the diagram, two approaches are shown:

- **Alonzo Church** → introduced the Lambda Calculus model of computation.
- **Alan Turing** → introduced the Turing Machine model.

Both models were proved to be equally powerful and capable of solving the same class of problems. This common conclusion is called the Church–Turing Thesis, which states that "effectively computable = Turing computable".

Thus, the diagram shows how two independent theories (Lambda Calculus and Turing Machine) merge into a single universal idea of computation.



Universal Turing Machine: A Universal Turing Machine (UTM) is a special type of Turing Machine that can simulate any other Turing Machine on any input. Normally, one Turing Machine can solve only one specific problem, but a UTM can execute the description of any Turing Machine and run it with its input. This makes it a general-purpose machine, just like our modern computers.

Attributes of UTM: It is a reprogrammable machine, which means we do not need to design a new machine for every problem.

It can simulate any other Turing Machine when given the description of that machine and its input.

UTM uses its tape to store three parts:

- Program area (description of another TM),
- State area (initial and current state),
- Data area (input string).

Working (Simple Idea):

UTM reads the encoded description of a Turing Machine (M) and its input.

It then follows the same steps as M would do on that input.

Finally, it produces the same output as that machine.

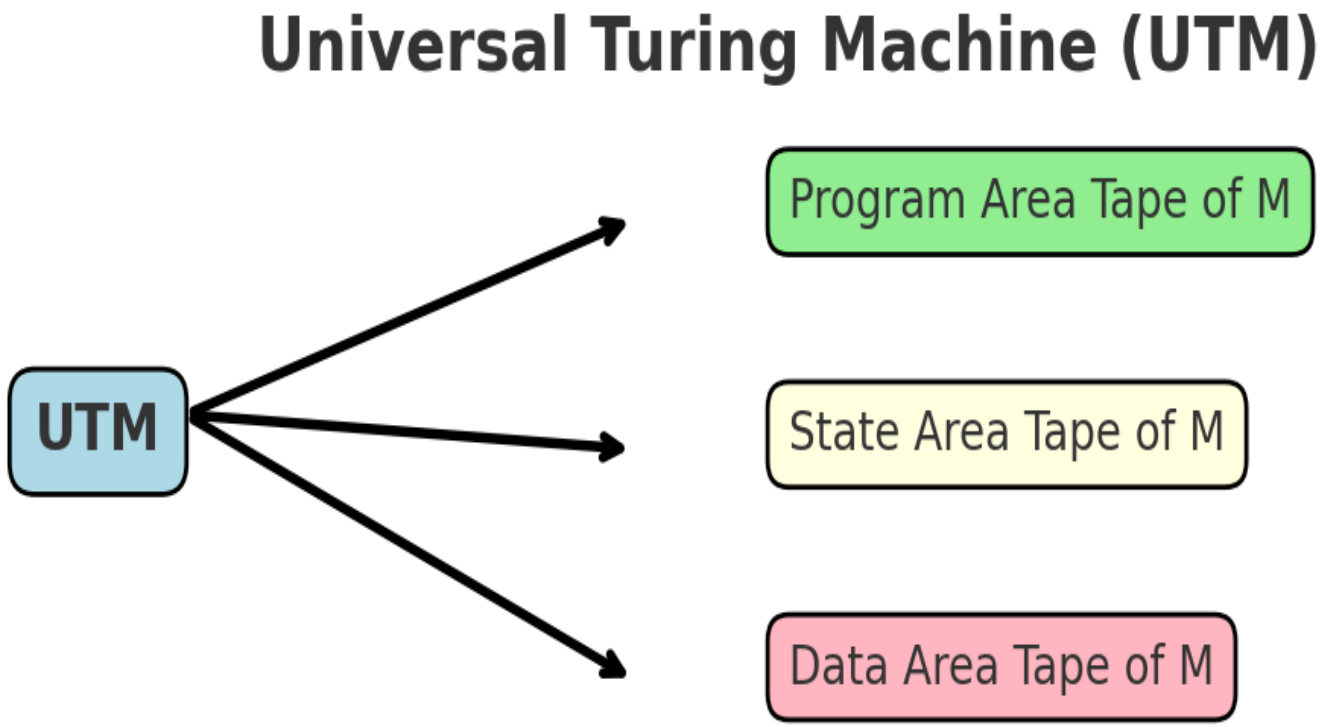
Example:

Suppose we have two Turing Machines:

- Machine 1 is encoded as #1#11#1,
- Machine 2 is encoded as ##1##1#1111.

If we give Machine 1's encoding and input string to UTM, it will work exactly as Machine 1 would, without needing a new machine.

- **Program Area Tape of M** → Encoded description of the machine
- **State Area Tape of M** → Current & initial state
- **Data Area Tape of M** → Input string



Recursive Enumerable Language: A Recursively Enumerable (RE) Language is a type of formal language in the theory of computation.

Definition: A Recursively Enumerable Language (also called Turing recognizable language) is a language for which there exists a Turing Machine (TM) that accepts all strings belonging to the language. If the input string is in the language, the TM will halt and accept it. If the string is not in the language, the TM may either reject it or run forever without halting.

This means that for RE languages, we can recognize when a string belongs to the language, but we cannot always decide when a string does not belong. Hence, every recursive (decidable) language is also an RE language, but not every RE language is recursive.

Consider the language:

L = { w | w is a valid program that halts on some input }.

• If w halts, the TM will accept it (halts).

• If w never halts, the TM may loop forever.

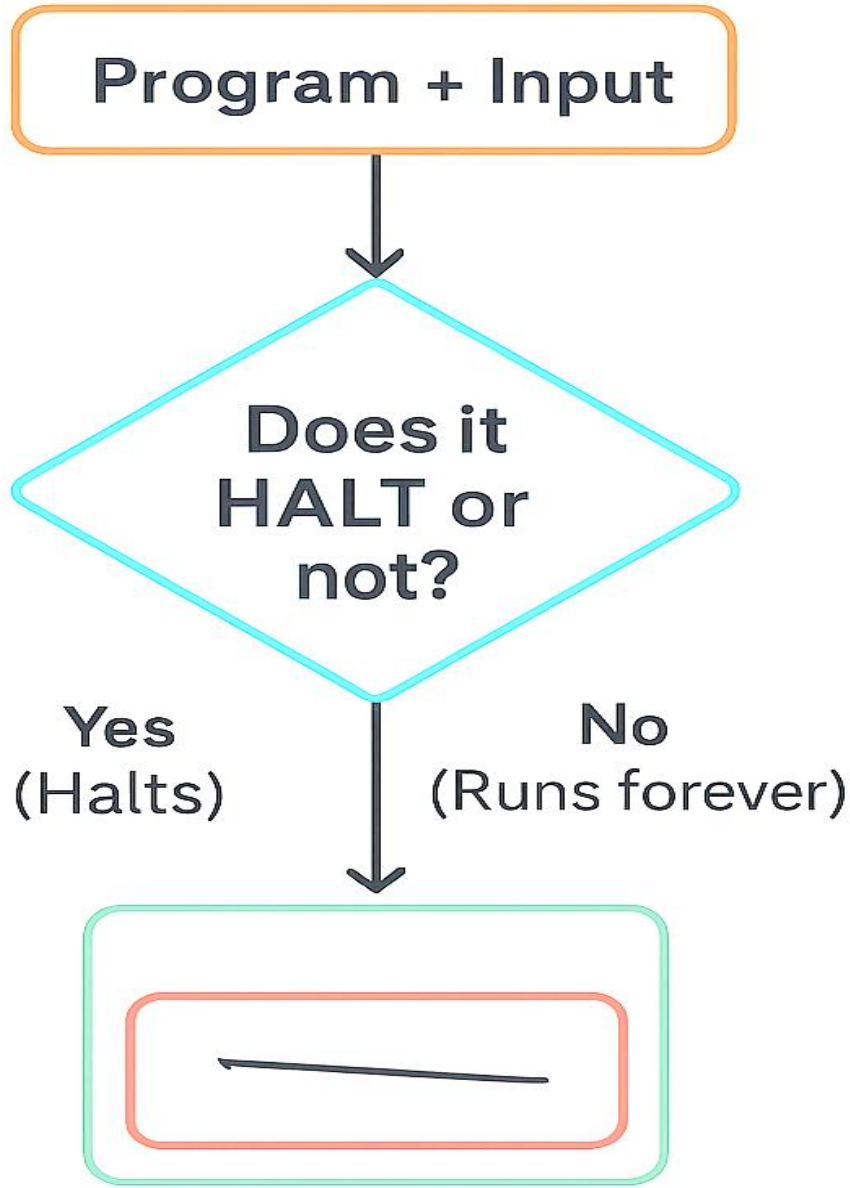
This language is recursively enumerable but not recursive, since we cannot always decide halting.

Halting Problem: The Halting Problem is one of the most important concepts in the theory of computation. It was introduced by Alan Turing in 1936. The problem asks: Given a program (or Turing Machine) and an input, can we decide whether the program will eventually stop (halt) or keep running forever?

If a program finishes its task and stops, we say it halts. If it keeps working in an infinite loop, we say it does not halt. Turing proved that there is no general algorithm that can solve this problem for all possible programs and inputs. In other words, the halting problem is undecidable.

The halting problem is significant because it shows the limitations of computers. Not all problems can be solved by machines, and some questions are fundamentally unsolvable. For example, suppose a program is written to check whether a number is prime. If the program is correct, it may halt after finishing the check. But if it has a logical mistake (infinite loop), it may never stop. There is no universal method to always decide this.

Thus, the Halting Problem proves that there are limits to what computers can do.


Program + Input
Does it HALT or not?
Yes
(Halts)
No
(Runs forever)

Complexity Hierarchy: Complexity hierarchy is the classification of computational problems into different classes (like P, NP, co-NP, PSPACE, EXPTIME) based on the time and space resources required to solve them.

1. P (Polynomial Time):

• Class of decision problems that can be solved in polynomial time by a deterministic Turing machine.
• These are considered efficiently solvable problems.
• Example: Sorting numbers, finding the shortest path (Dijkstra's algorithm).

2. NP (Nondeterministic Polynomial Time):

• Class of problems for which a solution (if given) can be verified in polynomial time, even if finding the solution may be hard.
• Example: Satisfiability problem (SAT), Hamiltonian cycle problem.
• Note: Every P problem is also NP (because if you can solve it fast, you can also verify it fast).

3. co-NP:

• Complement of NP.
• These are problems where the “No” answers can be verified in polynomial time.
• Example: "Is a formula unsatisfiable?" belongs to co-NP.

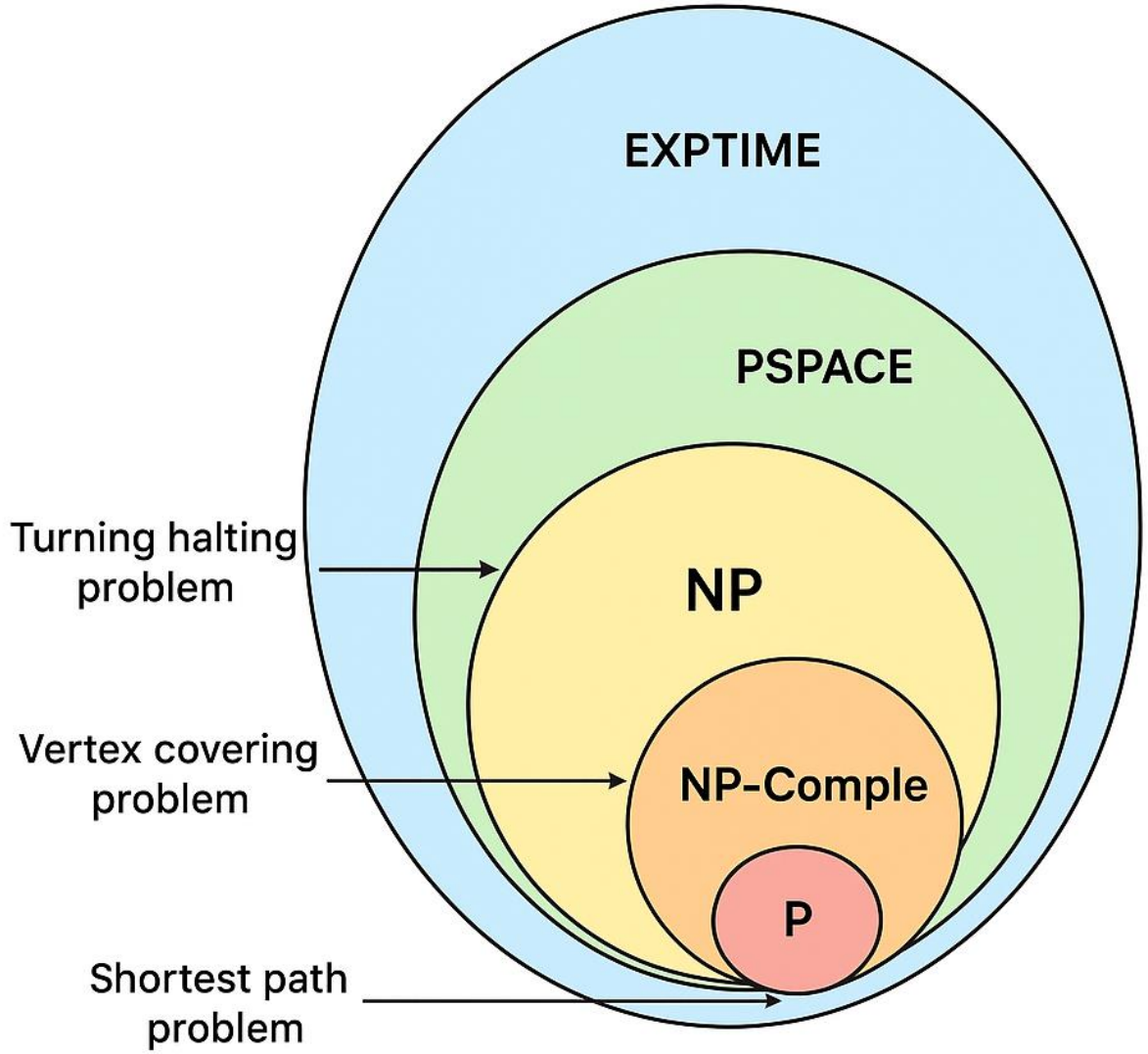
4. PSPACE (Polynomial Space):

• Class of problems that can be solved using a polynomial amount of memory (space), but may take longer time.
• PSPACE includes both P and NP problems.
• Example: Quantified Boolean Formula (QBF).

5. EXPTIME (Exponential Time):

• Class of problems that can be solved in exponential time.
• These are much harder than PSPACE problems.
• Example: Solving generalized chess or checkers.

Diagram of Complexity Hierarchy:



Big O Notation: Big O notation is a way to measure the efficiency of an algorithm. It tells us how the running time or memory of an algorithm increases when the size of input data becomes large. In simple words, it shows how fast an algorithm grows when the problem size increases.

Significance of Big O Notation:

1) Comparison of algorithms: It helps us to compare two or more algorithms and choose the most efficient one.
2) Scalability: It shows how the algorithm behaves when the input grows to a very large size.
3) Standard measure: It provides a common way to express performance in terms of input size.
4) Focus on growth rate: It ignores constants and lower-order terms and only considers the rate of growth.

Example:

- O(1) – Constant Time: Accessing an element from an array.
- O(log n) – Logarithmic Time: Binary Search in a sorted list.
- O(n) – Linear Time: Traversing all elements in an array (Linear Search).
- O(n log n) – Linearithmic Time: Merge Sort and Quick Sort (average case).
- $O(n^2)$ – Quadratic Time: Bubble Sort, Insertion Sort.
- $O(2^n)$ – Exponential Time: Recursive Fibonacci calculation.